

UI library manual

How to create UI elements to improve your interface

1.0 What is an “*UINode*”?

An UINode is a structure separated in some parts (position, size, inner, offset, ...) that are used to perform something.

They update in a pipeline in that order:

1. Sizes
2. Layout
3. World
4. Draw

1.1 UINode structure

One UINode has the following data:

- **Core:** Main data that keeps it's information
- **UI layer:** Says its layer in the UI
- **Assets:** Keeps it assets indexes
- **Position:** Has the X/Y coordinates information
- **Size:** Has the dimations of an UINode
- **Offset:** Contains the offsetvalue and pivot
- **Text:** All kinds of texts, spans and style are here
- **Scissor:** The draw and logic cutter information
- **Layout:** How the children and itself must be placed
- **Events:** Where all generic events are
- **Scroll:** Keeps the scroll data, like drag and force
- **Inner:** Here keeps all information that a child must use

1.2 UINode structure variables

Core:

- *parent:* The UINode reference to use as base
- *children:* The UINode children (who uses it has parent)
- *class:* A data to group UINodes
- *tag:* A tag to differ a node to another
- *id:* Node id value ("ref node X")
- *color:* A rgb color to apply in the UINode
- *alpha:* A value between 0-1 to define it's opacity
- *hovered:* if mouse is hover it (must be the top node)
- *visible:* Says wheater it is visible or not

Assets:

- *sprite_index*: The UINode sprite
- *image_index*: The subImage to use
- *sprite_set*: A set of sprites (used in sliders)

UI_layer:

- *id*: The UI_layer that the UINode is
- *group*: Which group it is
- *z_index*: The draw order in the group

Size:

- *width*: A struct with all kinds of width values
- *height*: A struct with all kinds of height values

Both width and height have:

- *raw*: The value passed by the user
 - *type*: Which type of value was passed
 - *inner*: The internal value of the size
 - *resolved*: The inner value plus padding
 - *outer*: The inner value plus padding and margin
- *padding*: Structure with raws and inner paddings
 - *margin*: Structure with raws and inner margins

Both padding and margin raw and inner values have:

- *top*: The top value
- *bottom*: The bottom value
- *left*: The left value
- *right*: The right value

Position:

Both x and y variables have:

- *raw*: The value passed by the user
- *type*: Which type of value was passed
- *inner*: The internal value of the coordinate
- *outer*: The inner value plus margin
- *final*: The outer value resolved (offset, scroll, ...)

Assets:

- *x*: The x coordinate offset value
- *y*: The y coordinate offset value
- *pivot*:
 - *x*: The amount of x pixels to be deslocated
 - *y*: The amount of y pixels to be deslocated

Text:

- *content*: The string passed by the user
- *wrap*: Content with automatic breaks (based on width)
- *span*: The rich array with text mutations
- *parsed*: The content/wrap string with span information
- *style*:
 - *font*: Font to apply when drawing
 - *color*: Color to apply when drawing text
 - *xscale*: The text x scale transform
 - *yscale*: The text y scale transform
 - *alpha*: Text's opacity (0-1)
- *layout*:
 - *halign*: The horizontal align
 - *valign*: The vertical align
 - *letters*: The amount of letters to be showed
 - *draw_mode*: If draws the raw content or wrap

Scissor:

- *enabled*: If the scissors must be applied or not
- *rect*: Struct with raw and inner values, each of them have:
 - *x*: Cut x coordinate
 - *y*: Cut y coordinate
 - *w*: Cut width
 - *h*: Cut height

Layout:

- *position*: If its RELATIVE or ABSOLUTE
- *justify_content*: The main axis flow
- *align_items*: The cross axis flow
- *flex_direction*: The children distribution
- *gap_x*: The gap in the x coordinate
- *gap_y*: The gap in the y coordinate

Events:

- *on_hover*: Calls if mouse enters the UINode
- *on_unhover*: Calls if mouse exits the UINode
- *can_click*: Check if is allowed to call *on_click*
- *on_click*: Calls if mouse is hover and mb_left released
- *on_input*: If an input was made (while in focus)
- *on_submit*: If is in focus and *enter* is pressed or blur
- *on_change*: If a **MAIN** change has occurred for *THAT* node

Offset:

- *x/y*: Which offset it has (UI_HALGIN / UIVALIGN)
- *pivot*:
 - *x/y*: The value (in pixels) that the UINode needs to deslocated

- **translate:**
 - *x/y*: The visual deslocation that the node has when drawing

Scroll:

- *flow*: Kind of scroll used (or noone)
- *enabled*: If a coordinate is enabled
- *value*: *The scroll offset values*
- *velocity*: A data to group UINodes
- *overflow*: The value of children overflowing (pixels)
- *must_hover*: *Defines whether the UINode must be hovered or only have the mouse inside its bounds*

Value, enabled, velocity and overflow have:

- *x*: X coordinate value
- *y*: Y coordinate value
- *force*: The mouse wheel force
- *drag*: The friction value to stop scroll
- *max_x/y*: Maximum value to x/y coordinate
- *min_x/y*: Minimum value to x/y coordinate

inner:

- *x*: *X outer value plus left padding*
- *y*: *Y outer value plus top padding*
- *width*: *Width resolved value*
- *height*: *Height resolved value*
- *context*: ANY kind of information that you want to guard

1.3 Types of UINode

By now, it has 9 types of UINodes

- | | | |
|----------|----------------|------------|
| • Panel | • Label | • Textbox |
| • Icon | • Checkbox | • Dropdown |
| • Button | • Radio button | • Slider |

Panel: They're a container (wheater or not with sprite), that can have children.

Icon: Simple images without big logic.

Button: interactables containers with texts.

Label: texts in the UI.

Checkbox: Buttons that may be on (true) or off (false).

Radio buttons: grouped checkboxes that just one can be on at a time.

Slider: an slideable container controled by a knob.

Textbox: Box that, when in focus, can recive inputs and show them.

Dropdown: container that displays options to be choosen when focused.

1.4 UINode especial variables

Some UINodes types has special variables. They're checkbox, radio button, dropdown, textbox and sliders

Checkbox:

- *value*: If it is activate or not

Radio button:

- *active*: If it is activate or not (only one active per group)
- *group*: The radio button group

Slider:

- *steps*: The amount of values that you can choose
- *is_holding*: If user is holding the knob (even out of it hitbox)
- *slider_values*:
 - *min_value*: The minimum value that a slider can have
 - *raw*: The step that the knob is (without min clamp)
 - *final*: Raw value with minimum clamp

Dropdown:

- *options*: All the options that the player can choose
- *is_open*: If dropdown is open
- *selected*:
 - *index*: The index of the selected value
 - *raw*: The raw value of the selected one
 - *display*: The string used to draw the choosen one
- *open*:
 - *separators*: If the options are drawed separated or not
 - *height*: The open area height
 - *Style*: How the open area will look like
 - *layout*: How open area will be placed/displayed

Textbox:

- *hide_text*: Hide the text ("*") if enabled
- *textbox_type*: How the textbox will be (linear or vertiacal)

2.0 How to create an UINode

First, to create an UINode, we will need to create a struct with the variables that we want to se, like width, scroll, offset, etc.

To define a variable in an UINode, you will need to put it right name, and then it value in a struct

2.1 All defining variables

They will be show at this format:

*variable name: {**allowed values**} (which variable it defines)*

- *parent:* {**UINode_reference**} (core.parent)
- *name:* {**any**} (core.name)
- *class:* {**any**} (core.class)
- *tag:* {**any**} (core.tag)
- *interactive:* {**Bool**} (core.interactive)
- *visible:* {**Bool**} (core.visible)
- *color:* {**Constant.Color**} (core.color)
- *alpha:* {**Real**} (core.alpha)

- *UI_layer:* {**UILayer enum**} (UI_layer.id)
- *layer_group:* {**String**} (UI_layer.group)

- *sprite:* {**Asset.GMSprite**} (asset.sprite_index)
- *image:* {**Real**} (asset.image_index)
- *sprite_set:* {**Struct**} (asset.sprite_set)

- *width:* {**Real, Percentage, Expression, Auto**} (size.width.raw)
- *height:* {**Real, Percentage, Expression, Auto**} (size.height.raw)

- *p_all:* {**Real, Percentage, Expression**} (size.padding.raw.*)
- *m_all:* {**Real, Percentage, Expression**} (size.margin.raw.*)

- p_x : {Real, Percentage, Expression} (size.padding.raw.left&right)
 - p_y : {Real, Percentage, Expression} (size.padding.raw.top&bottom)
 - p_* : {Real, Percentage, Expression} (padding for each side: top (t), right (r), bottom (b), left (l))
-
- m_x : {Real, Percentage, Expression} (size.margin.raw.left&right)
 - m_y : {Real, Percentage, Expression} (size.margin.raw.top&bottom)
 - m_* : {Real, Percentage, Expression} (margin for each side: top (t), right (r), bottom (b), left (l))
-
- x : {Real, Percentage, Expression} (position.x.raw)
 - y : {Real, Percentage, Expression} (position.y.raw)
-
- x_offset : {UI_HALIGN enum} (offset.x)
 - y_offset : {UI_VALIGN enum} (offset.y)
 - $Translate_x$: {Real, Percentage, Expression} (offset.translate.x.raw)
 - $Translate_y$: {Real, Percentage, Expression} (offset.translate.y.raw)
-
- $text$: {String} (text.content)
 - $span$: {Array*} (text.span)
-
- $font$: {Asset.GMFont} (text.style.font)
 - $font_color$: {Constant.Color} (text.style.color)
 - $font_xscale$: {Real} (text.style.xscale)
 - $font_yscale$: {Real} (text.style.yscale)
 - $font_alpha$: {Real} (text.style.alpha)
-
- $halign$: {fa_*} (text.layout.halign)
 - $valign$: {fa_*} (text.layout.valign)
 - $letters$: {Real} (text.layout.letters)
-
- $draw_mode$: {UINodeDrawMode enum} (text.layout.draw_mode)
-
- $enable_scissor$: {Bool} (scissor.enabled)
 - $scissor_rect$: {Struct {x, y, w, h} or UINode_reference} (scissor.rect.raw)

- *position*: {*UINodePosition* enum} (layout.position)
- *justify_content*: {*JUSTIFY_CONTENT* enum} (layout.justify_content)
- *align_items*: {*ALIGN_ITEMS* enum} (layout.align_items)
- *gap*: {*Real*} (layout.gap_x&y)
- *gap_x*: {*Real*} (layout.gap_x)
- *gap_y*: {*Real*} (layout.gap_y)
- *on_hover*: {*Function*} (events.on_hover)
- *on_unhover*: {*Function*} (events.on_unhover)
- *can_click*: {*Function*} (events.can_click)
- *on_click*: {*Function*} (events.on_click)
- *on_input*: {*Function*} (events.on_input)
- *on_submit*: {*Function*} (events.on_submit)
- *on_change*: {*Function*} (events.on_change)
- *scroll_flow*: {*UINodeScrollFlow* enum} (scroll.flow)
- *scroll_force*: {*Real*} (scroll.force)
- *scroll_drag*: {*Real* (<1)} (scroll.drag)
- *scroll_max_x*: {*Real*} (scroll.max_x)
- *scroll_max_y*: {*Real*} (scroll.max_y)
- *scroll_must_hover*: {*Bool*} {scroll.must_hover}
- *context*: {*Any*} (inner.context)

----- SPECIAL VARIABLES

- *on*: {*Bool*} (active, value)
- *group*: {*Any*} (group)
- *steps*: {*Real*} (steps)
- *value*: {*Real*} (slider_values.raw)
- *min_value*: {*Real*} (slider_values.min_value)

- *options*: {Array} (options)
- *selected*: {Real} (selected.index)
- *separators*: {Bool} (open.separators)
- *open_style*: {Function*} (open.style)
- *open_layout*: {Function*} (open.layout)
- *hide_text*: {Bool} (hide.enabled)
- *textbox_type*: {UINodeTextboxType enum} (textbox_type)

2.2 UINode declaration: special cases:

While declaring variables, there are tree cases (marked with “*”) that are **STRONGLY** recommended to use helper already created.

SPAN:

When creating the span array, **USE** these functions (more will be available in futher versions)

- create_font_span
- create_color_span
- create_alpha_span

DROPDOWN:

In a dropdown UINode, you can choose on the open struct the style and layout of it, to make sure nothing goes wrongs is recommended to use this functions:

- **Style:**
 - dropdown_color_style
 - dropdown_sprite_style
 - dropdown_gradient_style
- **Layout:**
 - dropdown_all_layout
 - dropdown_scroll_layout

3.0 Understanding UI_Layers

UI_Layers have 3 parts to understand how to use it.

First, it has **4 main layers**, the background, normal, foreground and overlay, being drawn in that order. Inside of each one, has groups.

In a group you have: It priority (bigger priority == more in front) and the nodes in the group.

In a group's node list, each of them have a z-index, that determinates when it will show, with the same logic of priority.

3.1 Creating groups in UI_Layers

By default, each UI_Layer has an group called "default", but you can add anothers group into a layer by using the function:

- `UI_layer_add_group`

4.0 Setting values after the creation of a node

After creating an UINode, you **CANNOT** set all variables directly, you'll have to use it setter. below has the list of all setters that you have to use instead of a directly attribution.

- | | | |
|--------------------------------|---------------------------------|---------------------------------|
| • <code>position.x.set</code> | • <code>size.padding.set</code> | • <code>scissor.set_rect</code> |
| • <code>position.y.set</code> | • <code>size.margin.set</code> | • <code>events.set</code> |
| • <code>size.width.set</code> | • <code>text.set_content</code> | |
| • <code>size.height.set</code> | • <code>text.set_span</code> | |

5.0 Extra functions

To help in some tasks, it was created some global functions to make small logics.

----- **ARRAYS**

- *empty_array*: returns if a array is empty or not
- *array_count*: Returns the amount of elements that is equals to the second argument
- *array_max*: Returns the biggest value in an number array (undefined if the array is empty)
- *array_min*: Returns the lowest value in an number array (undefined if the array is empty)

----- **STRUCTS**

empty_struct: Returns if a struct is empty or not

----- **GLOBAL**

- *color_multiply*: multiply the color by the passed factor
- *folder_length*: Returns the amount of files in a folder
- *file_name_validator*: Validades if a file name is malformed (with <>:\|?*)
- *is_asset*: Returns if the passed argument is asset (true) or not (false)

- *draw_gradient_rect_ext*: draws an gradient rectangle based on two colors (main and secondary) and a direction (*GRADIENT_DIRECTION* enum)

5.1 UINode functions

Like the global functions, the system has functions to it own logic:

- *get_UINode_by_id*: Get an UINode by a passed *NODE_REFERENCE*
- *UINode_get_id*: Get an UINode id
- *UINode_get_same_group*: Get all radios buttons in that group
- *UINode_add_children*: Put a new child in an UINode
- *UI_get_value_type*: Get the type of an UI system value
- *hide/show_all_UINodes*: Shows/hides every UINode
- *hide/show_UINode_by_class*: Shows/hides UINodes in that class
- *hide_UINode_except*: Shows/hides every UINode exept the one in that calss
- *destroy_UINode*: Destroy an UINode (and all it's children)
- *UINode_destroy_children*: Destroy all UINode children

5.2 Type of values

- **Any**: Can be any type
- **Real**: All kinds of numbers except bools (Reals, int34 and int64)
- **String**: Basically a sentence, sequence of letters
- **Bool**: True (1) or false (0)
- **Array**: A list of information
- **Struct**: A group of information stored in keys
- **Asset**: A reference to a GameMaker resource

- **Callable:** Something that can be executed (*functions / methods*)
- **Expression:** A string that starts with "=", it can be readed ("= 2 + 6" is equal to 8)
- **Percentage:** String that ends with "%" (normaly when used in UINodes it get the *parent metric * the percentage*)
-
- **Auto:** Makes it value depends on children (declared by "*auto*")
- **Node reference:** A string started with "ref node"
- **Undefined:** When a value is undefined
- **Unknown:** If the value don't match with anything it is *UNKNOWN*

6.0 Observations/warnings:

- The system **consumes** *keyboard_lastchar* and *keyboard_lastkey*, so any time you use them it will probably be empty.
- Using **AUTO** in **labels** can results in incosistences because the system is still in early stages. So its not recommended to use it by now.
- how the system is in the firsts steps, I may have some *bugs* or *optimization problems*. If you found any of them, please, report it to ***felipe.gamedev.gm@gmail.com***

Contact:

: felipe.gamedev.gm@gmail.com

UINode Framework

Created by Felipe Augusto - 2026 ©

**This project may be used and modified freely,
as long as proper credit is given to the original author.**

This project is provided as-is without warranty of any kind.